LMT

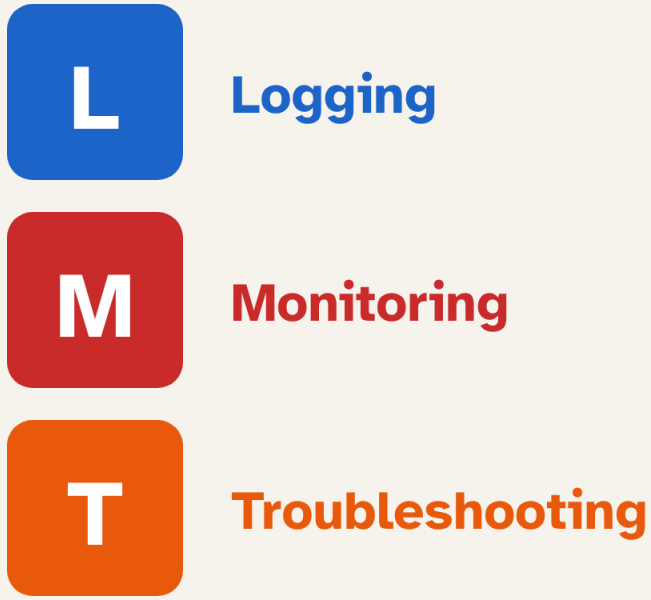
Logging, Monitoring, Troubleshooting

BIGNAMI

Tutto il corso, ridotto all'osso

ITS CyberSecurity - Jesi (AN) 2026/2027

prof. Luca Ferroni



A cosa serve ognuno

Logging

registra gli eventi in modo immutabile:
si aggiunge in coda, non si riscrive il passato

Monitoring

registra le metriche: numeri nel tempo
(CPU, memoria, disco, richieste al secondo)

Troubleshooting

cerca la causa radice (root cause) di un problema
usando i log e le metriche

I log sono davvero immutabili solo se escono dalla macchina: per questo si spediscono a Loki.

OBSERVABILITY

Logging + **Monitoring** + **Troubleshooting** = **Observability**

Capire cosa succede dentro un sistema guardando quello che produce:
log ed eventi, metriche (e, nelle architetture più grandi, le tracce).

- sapere in ogni momento cosa succede nella rete e nei servizi
- correlare: stesso istante, stesso host, log e metriche affiancati in Grafana
- arrivare alla root cause di un guasto in tempi rapidi

Il monitoring ti dice che qualcosa non va; l'observability ti aiuta a capire perché.

PARTE 1

LOGGING

Registrare gli eventi e conservarli **a norma di legge**

Dove finiscono i log sulla macchina

Quattro posti da conoscere

systemd-journald

binario, si legge solo con journalctl
è il default delle distribuzioni recenti

```
$ journalctl -u ssh -f
```

syslog (rsyslog)

testo, in /var/log/syslog e /var/log/auth.log
si legge con cat, grep, tail

```
$ tail -f /var/log/auth.log
```

file dell'applicazione

in /var/log o dove decide l'applicazione
es. Apache, Nginx, Postfix

```
$ tail -f /var/log/apache2/access.log
```

log di Docker

stdout e stderr del container
2>&1 per passarli entrambi a grep

```
$ docker logs -f grafana 2>&1 | grep -i error
```

Come si centralizzano i log

Quattro architetture: nel corso usiamo Loki

	raccolta	storage	visualizzazione	nota
syslog	inoltro al server rsyslog UDP/TCP 514, TLS 6514	file di testo	grep, tail	la più semplice: c'è già ovunque
ELK Elastic Stack	Beats o Logstash	Elasticsearch, indice full-text	Kibana	ricerche potenti, ma pesante in RAM e disco
Graylog	GELF, syslog, Beats	OpenSearch (più MongoDB)	web di Graylog	pronto all'uso, con stream e allarmi
Loki nel corso	Alloy	indicizza solo le etichette	Grafana	leggero: per questo lo usiamo

Un file di log e tre righe di shell

logging/sshd_classic.log dal repository del corso

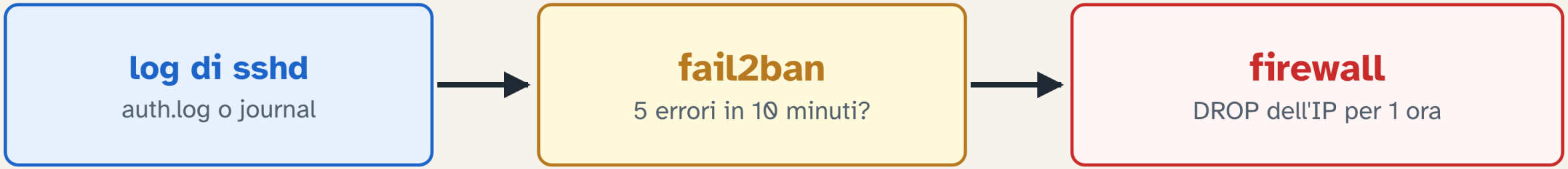
```
Sep  7 18:43:41 srv sshd[5319]: Failed password for pippo from 10.0.0.7 port 47004 ssh2
Sep  8 03:11:00 srv sshd[7438]: Failed password for invalid user admin from 185.22.1.9 port 46374 ssh2
Sep 28 02:49:02 srv sshd[2208]: Accepted password for deploy from 45.9.14.207 port 42216 ssh2
```

```
$ grep -c 'Failed password' sshd_classic.log
49
$ grep 'Failed password' sshd_classic.log | grep -o 'from [0-9.]*' | sort | uniq -c | sort -rn | head -3
     36 from 45.9.14.207
     10 from 185.22.1.9
      3 from 10.0.0.7
$ grep Accepted sshd_classic.log | grep 45.9.14.207
Sep 28 02:49:02 srv sshd[2208]: Accepted password for deploy from 45.9.14.207 ...
```

36 password sbagliate e poi un accesso riuscito: il brute force è andato a segno.

fail2ban: dal log all'azione

Legge i log e banna chi sbaglia troppe volte




```
# /etc/fail2ban/jail.local
[sshd]
enabled  = true
maxretry = 5
findtime = 10m
bantime  = 1h
```

```
$ sudo fail2ban-client status sshd
   |- Currently banned: 1
   `- Banned IP list: 45.9.14.207
```

Nel nostro log il brute force parte con 8 errori di fila:

sarebbe stato bannato prima dell'accesso riuscito.

Lezione 2: "fail2ban" era il nome finto del miner. Controllate sempre il percorso del programma.

Il firewall che scrive nel log

iptables -j LOG: ogni pacchetto che corrisponde finisce nel log del kernel

```
$ sudo iptables -I INPUT -p tcp --dport 22 -j LOG --log-prefix "SSH-IN: "
$ sudo journalctl -k -f | grep SSH-IN
Oct 28 10:15:02 srv kernel: SSH-IN: IN=eth0 OUT= SRC=45.9.14.207 DST=203.0.113.10
    ... PROTO=TCP SPT=42216 DPT=22 SYN
```

- SRC e DST sono il livello 3, SPT e DPT il livello 4
- -j LOG non blocca: scrive e passa alla regola successiva
- -I la mette in cima, prima delle regole che accettano o scartano
- Sui sistemi recenti c'è nftables: stessa idea, sintassi diversa

PARTE 2

MONITORING

Misurare lo stato nel tempo

Classificazione degli indicatori chiave

Cosa misurare, a seconda di cosa si osserva

Apparati di rete

router, switch, firewall

- throughput
- errori sulle interfacce
- latenza
- CPU e RAM

di solito via SNMP, senza agent

Server

GNU/Linux, Windows

- utilizzo CPU
- RAM
- I/O disco
- spazio disco
- processi attivi

con un agent (Alloy, node exporter)

Servizi e applicazioni

web, database, API

- uptime
- status code HTTP
- tempi di risposta
- porte aperte

controllati anche da fuori (curl, nc)

Le tre scelte del monitoraggio

Agent o agentless

agent

un programma sulla macchina

(Alloy, node exporter)

agentless

si osserva da fuori

(SNMP, ping)

Push o pull

push

l'agent spedisce al server:

va bene dietro un router

pull

il server va a prendere:

deve raggiungere la macchina

Retention

cos'è

per quanto tempo tengo i dati

prima di cancellarli

costo

più giorni = più disco

es. 15 giorni, 1 mese

Nel corso: agent (Alloy) che fa push, perché i PC hanno IP privati.

SNMP per gli apparati di rete

Switch, router, stampanti, UPS: niente da installare. UDP 161 (domande) e 162 (trap)

	v1 / v2c	v3
accesso	community (es. public)	utente e password
cifratura	nessuna: tutto in chiaro	sì (livello authPriv)
quando	rete fidata, sola lettura	sempre, se l'apparato lo supporta

```
$ snmpget -v2c -c public 192.168.1.1 1.3.6.1.2.1.1.3.0
```

```
$ snmpget -v3 -l authPriv -u monitor -a SHA -A <pwd> -x AES -X <pwd> 192.168.1.1 1.3.6.1.2.1.1.3.0
```

La community di v1 e v2c viaggia in chiaro: chi la intercetta legge tutto.

Metriche, eventi e alerting

Metrica
un numero nel tempo

`CPU all'87% alle 10:15`

Evento
un fatto puntuale

`10:15:02 servizio riavviato`

Alert
una situazione anomala

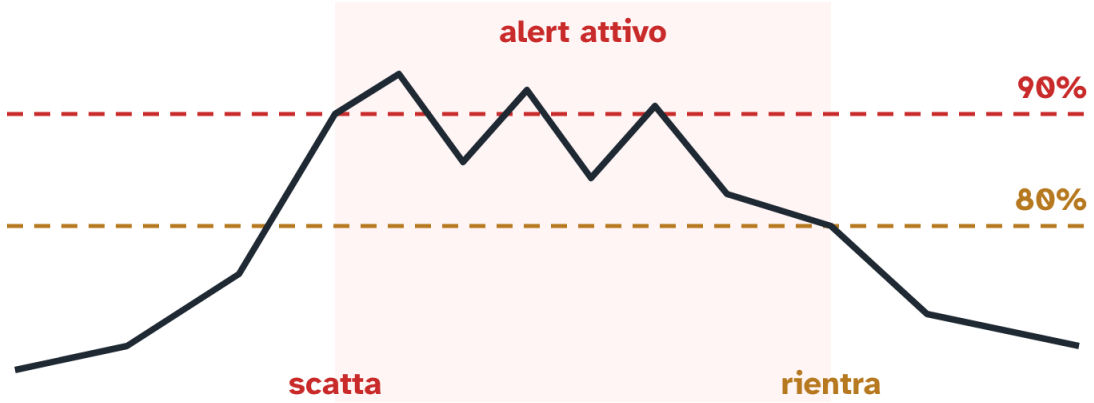
`CPU oltre il 90% da 5 minuti`

Soglie (thresholds)

statica: un valore fisso, es. disco oltre il 90%

dinamica: rispetto al normale, es. il triplo del solito

Isteresi: scatta sopra il 90%, rientra solo sotto l'80%



Senza isteresi si accenderebbe e spegnerebbe a ogni oscillazione.

Severity

Info	da registrare, nessuna azione
Warning	da guardare prima che peggiori
High	da risolvere presto
Critical	servizio fermo: subito

Alert fatigue

troppi falsi positivi: nessuno legge più gli alert, nemmeno quello vero.

Un alert solo se serve un'azione.

Dalla sorgente al grafico

Lo stesso flusso, per i log e per le metriche

Raccolta collection	**Parsing** e normalizzazione	**Storage** con una retention	**Visualizzazione** e interrogazione
prende i dati dove nascono	stesso formato e nomi: etichette, unità, orari	conserva i dati per un tempo deciso	grafici, ricerche, dashboard
LOG			
rsyslog, Alloy, Beats	**Logstash, gli stage di Alloy**	**Loki, Elasticsearch**	**Grafana, Kibana**
METRICHE			
exporter, Telegraf, agent Zabbix, SNMP	**etichette host e job, unità di misura**	**Prometheus, InfluxDB**	**Grafana, Zabbix**

L'alerting parte dallo storage: è una query che gira da sola, ogni pochi secondi.

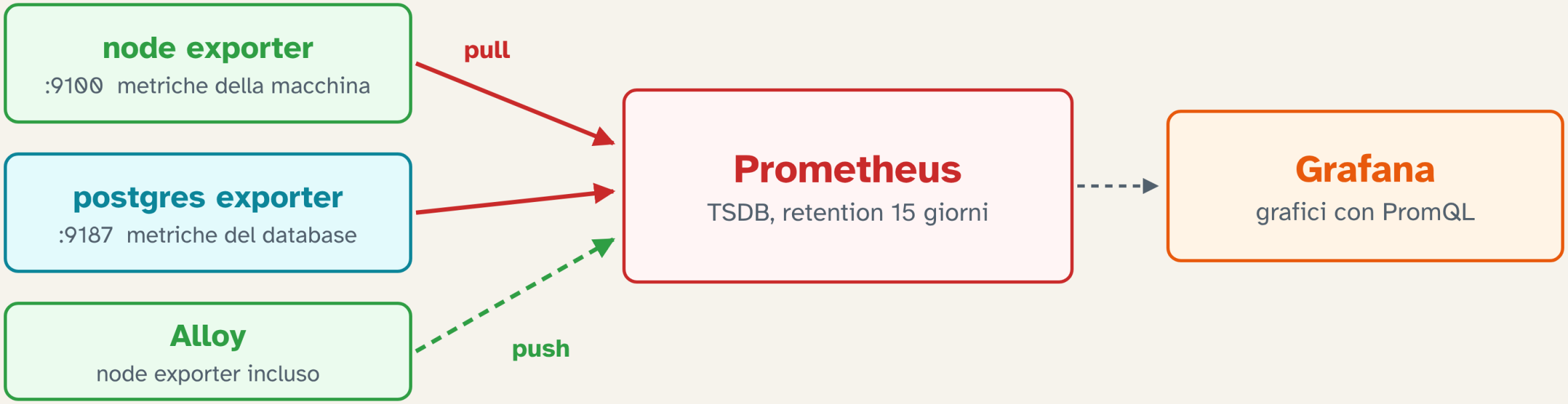
Gli strumenti di monitoraggio più usati

Nel corso usiamo Prometheus: è lo standard e si integra con Alloy e Grafana

	come raccoglie	dove salva	in una riga
Zabbix	agent proprio, e agentless: SNMP, IPMI, ping	database relazionale (PostgreSQL, MySQL)	tutto in uno: raccolta, web, trigger e allarmi
TIG Telegraf InfluxDB Grafana	Telegraf: agent con molti plugin, in push	InfluxDB, una TSDB	come il nostro stack, ma in push nativo
Prometheus	pull da /metrics degli exporter; push con remote write	TSDB propria	PromQL e Alertmanager; lo standard per i container
Netdata	agent su ogni macchina, una metrica al secondo	sulla macchina stessa (o inoltra a un parent)	zero configurazione, dashboard già pronta
PCP Performance Co-Pilot	demone pmcd, un agent per sottosistema	archivi di pmlogger	analisi delle prestazioni, anche a posteriori

Prometheus

Un database a serie temporali (TSDB) che si interroga con PromQL




```
up                                                          1 se il target risponde
node_load1                                                  carico medio dell'ultimo minuto
100 - avg by (host) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100   CPU usata in %, per host
```

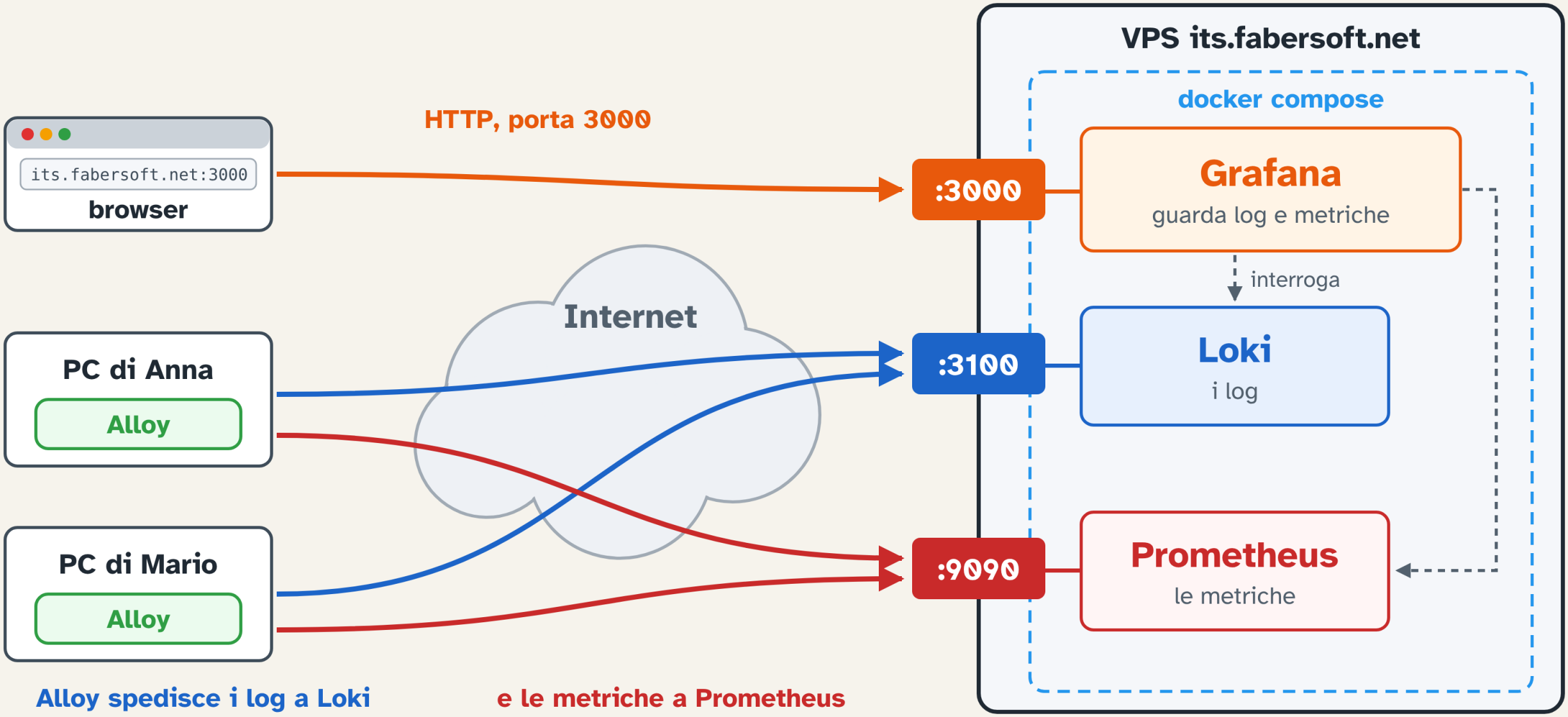
PARTE 3

COME DEPLOYARE

Un'architettura di rete per log e metriche

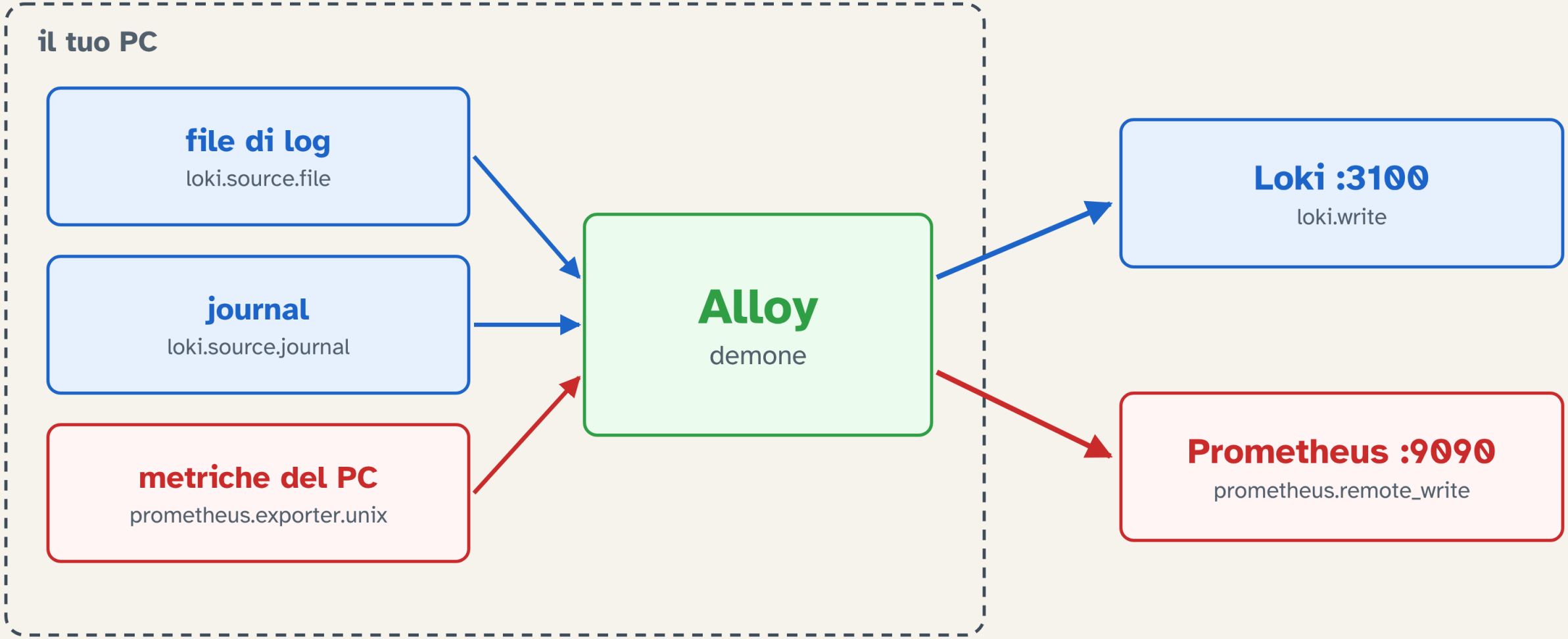
Il quadro completo

Chi guarda log e metriche, e chi li manda



Alloy: il demone sul PC

Un solo programma, client di Loki e di Prometheus



Gira sempre: come servizio systemd, o in un container con restart: unless-stopped.

Docker: un servizio in una riga

Immagine dal registry, container che gira, porte e log

```
$ docker run -d --name grafana -p 3000:3000 grafana/grafana
$ docker ps
$ docker logs -f grafana 2>&1
```

- **-d** gira in background
- **-p** porta del PC : porta del container
- **2>** anche lo stderr, così grep vede gli errori

Docker Compose: più container insieme

Si lancia nella cartella che contiene compose.yml

1. **`docker compose up -d`**

   crea e avvia tutti i servizi del file

2. **`docker compose down`**

   ferma e cancella i container: i volumi con i dati restano

3. **`docker compose down -v`**

   cancella anche i volumi: i dati sono persi

Come con docker: docker compose ps e docker compose logs -f <servizio>.

Da docker run a compose.yml

La stessa riga, scritta in un file

```
$ docker run -d --name grafana -p 3000:3000 grafana/grafana
```



```
services:
  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
    restart: unless-stopped
```

← --name

← l'immagine

← -p

← in più: riparte da solo,
anche dopo un riavvio

compose.yml del server

Loki, Grafana e Prometheus sul VPS, nello stesso file (continua a destra)

```yaml
services:
  loki:
    image: grafana/loki:3.7.0
    command: -config.file=/etc/loki/config.yml
    volumes:
      - ./loki/config.yml:/etc/loki/config.yml:ro
      - loki-data:/loki
    ports: ["3100:3100"]
    restart: unless-stopped
  grafana:
    image: grafana/grafana:13.2.4
    volumes:
      - grafana-data:/var/lib/grafana
    ports: ["3000:3000"]
    restart: unless-stopped
```

```yaml
  prometheus:
    image: prom/prometheus:v3.15.0
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --storage.tsdb.path=/prometheus
      - --storage.tsdb.retention.time=15d
      - --web.enable-remote-write-receiver
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    ports: ["9090:9090"]
    restart: unless-stopped
volumes:
  loki-data:
  grafana-data:
  prometheus-data:
```

Porte esportate: **:3000 Grafana** **:3100 Loki** **:9090 Prometheus** retention 15 giorni, accetta il push

compose.yml del PC

Alloy, che spedisce a Loki e a Prometheus

```yaml
services:
  alloy:
    image: grafana/alloy:v1.20.1
    hostname: mario-rossi
    command:
      - run
      - --server.http.listen-addr=0.0.0.0:12345
      - --storage.path=/var/lib/alloy/data
      - /etc/alloy/config.alloy
    volumes:
      - ./alloy/config.alloy:/etc/alloy/config.alloy:ro
      - /var/log:/var/log:ro
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - alloy-data:/var/lib/alloy/data
    ports: ["127.0.0.1:12345:12345"]
    restart: unless-stopped
volumes:
  alloy-data:
```

← diventa l'etichetta host

← config.alloy dice dove spedire log e metriche

← i log da leggere

← le metriche del PC

← solo la pagina di stato, visibile solo dal PC

Nessuna porta in ingresso:
è Alloy che apre la connessione.

Tenere d'occhio il disco

Log e metriche crescono ogni giorno

Misurare

```
$ df -h /
$ docker system df
$ sudo du -sh /var/lib/docker/volumes/*
$ journalctl --disk-usage
```

Limitare (retention)

```
# Prometheus (compose.yml)
--storage.tsdb.retention.time=15d

# Loki (config.yml)
limits_config:
  retention_period: 744h
compactor:
  retention_enabled: true
  delete_request_store: filesystem

# journald (/etc/systemd/journald.conf)
SystemMaxUse=500M
```

Retention giusta =

quanto indietro devo poter guardare

e quanto disco ho a disposizione.

Disco pieno: Loki e Prometheus smettono di scrivere, e si perdono proprio i dati del guasto.

PARTE 4

TROUBLESHOOTING

Trovare la causa, usando log e metriche

Strumenti per livello

Dal client si sale da 3 a 7; sul server si guarda chi ascolta e cosa dice

	dal client	sul server
7 applicazione	`curl -I http://host:3000`	`docker compose logs -f grafana` `journalctl -u nome-servizio`
4 porta	`nc -zv host 3000`	`sudo ss -lptn | grep 3000` `sudo tcpdump -ni any port 3000`
3 IP	`ping -c 3 host`	`ip a` `sudo tcpdump -ni any icmp`

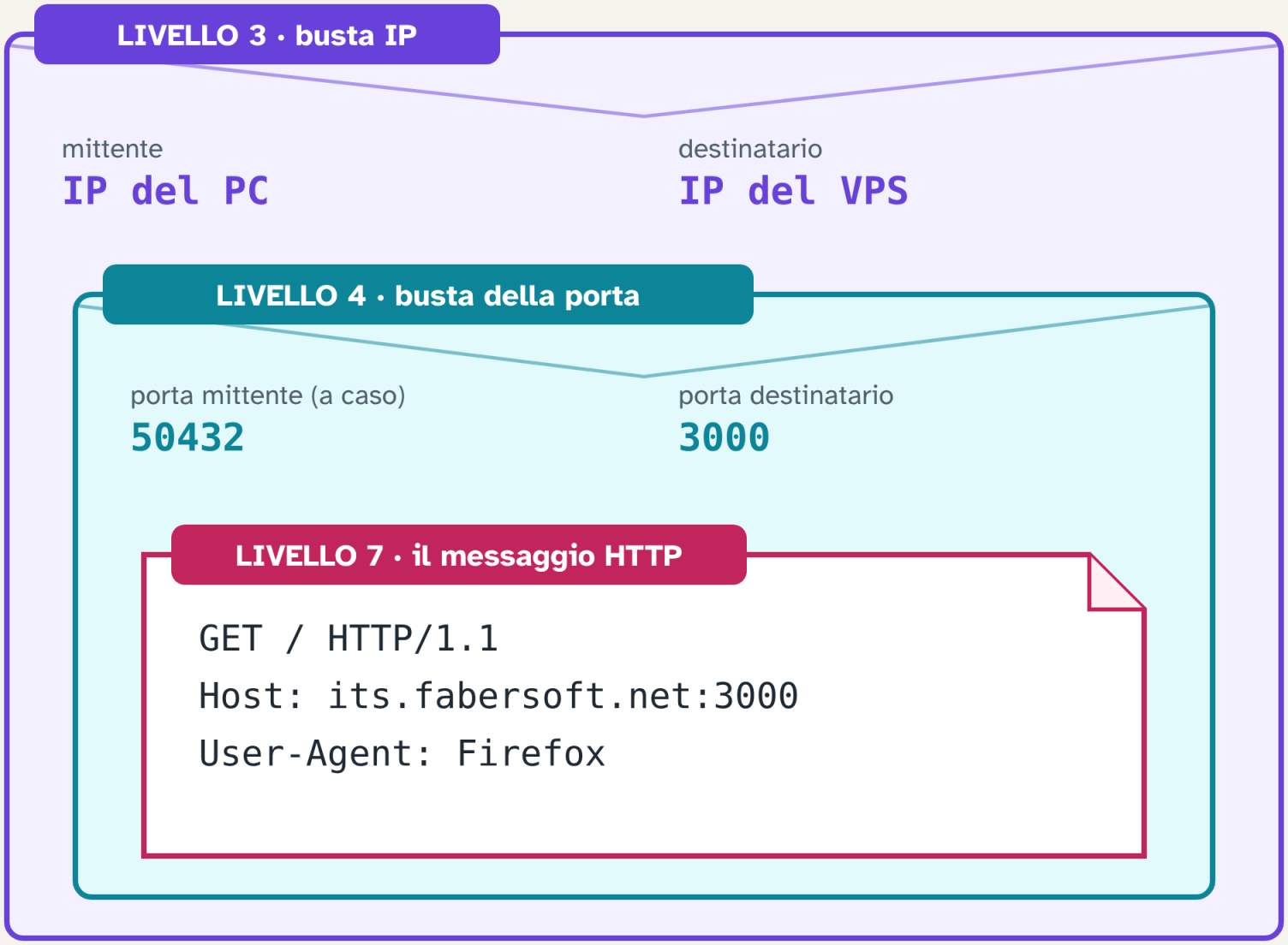
APPENDICE

RETE E WEB

Buste, URL e HTTP

Incapsulamento: tre buste

Il browser chiede una pagina a Grafana sul VPS



Chi apre quale busta

3 · indirizzo IP — `ping`

I router lungo Internet: portano il pacchetto al VPS.

4 · porta — `nc -zv`

Il sistema operativo del VPS: consegna al programma in ascolto.

7 · messaggio — `curl`

Solo il programma, qui Grafana: legge la richiesta e risponde.

Uniform Resource Locator (URL)

Fonte: it.wikipedia.org/wiki/Uniform_Resource_Locator

In giallo i separatori: :// : / ? #



<proto>

il protocollo di livello 7: http o https

finisce con ://

<fqdn>

il nome completo del server: il DNS

lo traduce nell'indirizzo IP (livello 3)

<port>

la porta (livello 4), dopo i :

si omette se è quella standard: 80 o 443

<path>

la risorsa sul server, dopo la prima /

è quello che finisce nei log del web server

<querystring>

parametri per il server, dopo il ?

nome=valore, più parametri uniti da &

<anchor>

un punto della pagina, dopo il #

resta nel browser: non arriva al server

HTTP/1.1: richiesta e risposta

La pagina di Wikipedia sull'URL, e le risposte da riconoscere

richiesta (dal client)

```
GET /wiki/Uniform_Resource_Locator?uselang=it HTTP/1.1
Host: it.wikipedia.org
User-Agent: curl/8.5.0
Accept: */*
```

riga vuota

La riga vuota chiude la richiesta.

#cite_note-2 non c'è: l'anchor resta nel browser.

risposta (dal server)

```
HTTP/1.1 200 OK
Date: Wed, 28 Oct 2026 10:15:02 GMT
Content-Type: text/html; charset=UTF-8
Content-Language: it

<!DOCTYPE html>
<html class="client-nojs" lang="it" dir="ltr">
... (il resto della pagina)
```

riga vuota: poi arriva il body

302 Found: vai altrove

```
$ curl -I http://its.fabersoft.net:3000
HTTP/1.1 302 Found
Location: /login
```

Grafana risponde e rimanda alla pagina di login.

404 Not Found: non esiste

```
GET /wiki/Pagina_che_non_esiste HTTP/1.1
HTTP/1.1 404 Not Found
Content-Type: text/html; charset=UTF-8
```

Il server risponde, ma quel path non c'è.

Qualsiasi codice vuol dire che il livello 7 risponde: 2xx ok, 3xx altrove, 4xx errore del client, 5xx del server.